PTL AI — Tutor Me Feature

Development Automation Plan with Claude Code

Sprint-by-Sprint Implementation Guide for the Development Team

---

Unreal Engine 5 + C++/Blueprint • GitHub • Claude Code

Date:	29 March 2026
Author:	Thiruvengadam
Tech Stack:	Unreal Engine 5 / C++ / Blueprint
Repository:	GitHub (Private)

1. How Claude Code Accelerates Your Dev Team

Claude Code is a command-line AI coding agent that lives inside each developer's terminal. It reads your codebase, understands your project's conventions, writes code, runs builds, creates pull requests, and reviews code — all while following your project's specific rules defined in a CLAUDE.md file. Here is how it works alongside your Unreal Engine team:

1.1 The Developer + Claude Code Workflow

Each developer on your team will use Claude Code as a pair programming partner. The workflow integrates directly with your existing GitHub process:

1. **Developer opens terminal in the PTL AI repo:** Claude Code reads the CLAUDE.md file and understands the project structure, coding conventions, module boundaries, and the Tutor Me feature requirements.
2. **Developer gives a task in natural language:** e.g., "Implement the Tutor Me button widget for the video player overlay using the design from the PRD."
3. **Claude Code writes the code:** It generates C++ classes, Blueprint scaffolding, API integration code, and UMG widget implementations — following your project's conventions.
4. **Developer reviews and iterates:** The developer reviews the output, asks Claude Code to adjust, refactor, or add tests. It's a conversation.
5. **Claude Code creates a PR:** When the code is ready, Claude Code creates a GitHub pull request with a proper description, linking to the relevant sprint task.
6. **Team reviews and merges:** Other team members review the PR (they can also use Claude Code to review PRs). Once approved, it merges into the feature branch.

1.2 What Claude Code Can and Cannot Do for Unreal Engine

Claude Code CAN Do	Claude Code CANNOT Do
Write C++ header and source files (.h / .cpp) for UE5 classes, components, subsystems, and plugins	Visually design Blueprint graphs (devs create Blueprints in-editor, Claude Code writes the C++ base classes)
Generate UMG widget C++ classes and layout code for the Tutor Me UI	Compile Unreal Engine projects directly (devs compile in-editor or via UnrealBuildTool)
Write REST API client code (HTTP module) for Smart Content Studio data retrieval	Design 3D assets, animations, or materials for Miss Lily avatar (artists handle this)
Generate data models, serialization/deserialization code, and database schema	Run the Unreal Editor or test gameplay visually (devs do play-testing)

Write unit tests using UE5 Automation Framework	Configure Unreal project settings (.uproject, .ini files should be done carefully by devs)
Create GitHub PRs, review code, write documentation, generate API specs	Replace the dev team — it accelerates them, it doesn't replace their judgment
Write backend API code (Node.js/Python) for the evaluation engine and session management	Deploy to production or manage cloud infrastructure (DevOps handles this)

2. Project Architecture for Claude Code

The Tutor Me feature spans three layers. Claude Code will work across all three, with each developer focusing on their layer:

2.1 Module Breakdown

Module	Tech	Dev Role	Claude Code Assists With
UE5 Frontend	C++ / UMG / Blueprint	UE5 Developer	Widget classes, state machines, UI logic, animation triggers
Backend API	Firebase Cloud Functions / Firestore	Backend Developer	Cloud Functions, Firestore models, session management, evaluation engine
AI Evaluation	Python + LLM API	AI/ML Engineer	Prompt engineering, evaluation logic, scoring algorithms, remediation strategies
Smart Content Studio	Firestore + Cloud Functions	Full-stack Dev	New Firestore collections, segment data schema, Cloud Functions
Miss Lily Integration	UE5 + TTS/STT APIs	UE5 Developer	Avatar controller C++ class, lip-sync integration, emotion state machine

2.2 Repository Structure

Claude Code works best with a clear folder structure. Here is the recommended layout for the Tutor Me feature module within your existing PTL AI repository:

```
PTL-AI/
├── .claude/
│   ├── CLAUDE.md                    ← Project-wide rules for Claude Code
│   ├── commands/
│   │   ├── generate-ue-class.md     ← Custom command: scaffold UE5 class
│   │   ├── generate-api-endpoint.md ← Custom command: scaffold API route
│   │   ├── review-pr.md             ← Custom command: AI code review
│   │   └── write-test.md            ← Custom command: generate UE5 test
│   └── settings.json                ← Claude Code project settings
├── Source/TutorMe/              ← UE5 C++ module
│   ├── Public/                  ← Header files
```

```
│   └── Private/                  ← Source files
├── Content/TutorMe/              ← UE5 assets (Blueprints, UMG, materials)
├── functions/                    ← Firebase Cloud Functions
│   ├── src/tutorMe/
│   │   ├── endpoints/            ← Cloud Function handlers
│   │   ├── services/             ← Business logic
│   │   └── models/               ← Firestore schema types
│   ├── tests/
│   └── firestore.rules           ← Security rules
└── AI/TutorMe/                   ← AI evaluation engine
    ├── prompts/
    ├── evaluation/
    └── remediation/
```

3. CLAUDE.md — The Project Intelligence File

The CLAUDE.md file is the single most important file for Claude Code integration. It tells Claude Code everything it needs to know about your project. Place this at .claude/CLAUDE.md in your repository root. Here is the complete file your team should use:

```
# CLAUDE.md — PTL AI Tutor Me Feature

## Project Overview
PTL AI is an immersive AI-powered learning platform for children aged 8-16,
built on Unreal Engine 5. The 'Tutor Me' feature adds interactive AI tutoring
sessions triggered from Lesson Shorts videos. The AI tutor (Miss Lily) teaches
segment-by-segment, assesses student understanding, and adapts teaching
approach.

## Tech Stack
- Game Engine: Unreal Engine 5.4+ (C++ and Blueprint)
- UI Framework: UMG (Unreal Motion Graphics) with C++ base classes
- Backend API: Node.js with Express (or FastAPI for AI endpoints)
- Backend: Firebase (Firestore, Cloud Functions, Firebase Auth, Cloud Storage)
- AI/LLM: Anthropic Claude API for evaluation, Google Cloud STT for voice
- Hosting: Firebase Hosting + Cloud Functions for serverless API
- Repository: GitHub with branch protection on main
- CI/CD: GitHub Actions for backend, manual builds for UE5

## Coding Conventions
- C++ follows Unreal Engine coding standard (prefix: U for UObject, A for
AActor,
  F for structs, E for enums, I for interfaces, T for templates)
- All new C++ classes MUST include UCLASS/USTRUCT/UENUM macros
- Use UPROPERTY(EditAnywhere/BlueprintReadWrite) for Blueprint-exposed
variables
- Use UFUNCTION(BlueprintCallable) for functions accessible from Blueprint
- Backend uses Firebase Cloud Functions with /api/v1/ prefix
- Firestore collections follow camelCase naming (tutorSessions, videoSegments)
- All API responses use { success: bool, data: {}, error: string } format
- AI prompts stored as .md files in AI/TutorMe/prompts/

## Module Boundaries
- Source/TutorMe/ = UE5 C++ classes (UI, state machine, avatar controller)
- Backend/TutorMe/ = Firebase Cloud Functions (functions/, firestore rules)
- AI/TutorMe/ = Evaluation engine, prompts, scoring logic
```

```
- NEVER mix UE5 C++ with backend code in the same PR
- Each PR must target exactly ONE module


## Key Data Flow
1. Student taps 'Tutor Me' → UE5 calls Cloud Function
/api/v1/videos/{id}/segments
2. Cloud Function fetches from Firestore (videoSegments collection) → returns
segment[]
3. UE5 displays segment via UTutorMeSessionWidget
4. Student responds → UE5 sends POST /api/v1/tutor-sessions/{id}/evaluate
5. Backend calls AI evaluation engine → returns score + feedback
6. If fail: Backend calls /remediate → returns adapted teaching content
7. After all segments: POST /final-evaluate → returns mastery score


## PR and Commit Rules
- Branch naming: feature/tutor-me/{module}-{description}
- Commit messages: [TutorMe][Module] Description
- Every PR must have: description, test plan, screenshots (for UI changes)
- PRs require 1 approval before merge
- Squash merge to feature/tutor-me, then merge to develop
```

4. Custom Claude Code Commands for Your Team

Custom commands let any developer on your team type a slash command (like /generate-ue-class) and have Claude Code perform a complex, standardised task. Here are the commands to create in .claude/commands/:

4.1 /generate-ue-class — Scaffold a UE5 C++ Class

When a developer types: /generate-ue-class UTutorMeSessionManager UGameInstanceSubsystem

Claude Code will generate a properly formatted .h and .cpp file pair with all UE5 macros, includes, and boilerplate. The command file contents:

```
Generate a new Unreal Engine 5 C++ class.
Arguments: $ARGUMENTS (ClassName BaseClass)

Create both .h and .cpp files in Source/TutorMe/Public/ and Private/.
Follow UE5 coding standard. Include: copyright header, #pragma once,
generated.h include, UCLASS macro with appropriate specifiers,
constructor, BeginPlay, and any virtual overrides from the base class.
Add TODO comments where implementation logic should go.
```

4.2 /generate-api-endpoint — Scaffold a Backend Route

Creates a complete API endpoint with route handler, validation, service layer, and test file:

```
Generate a new API endpoint for the Tutor Me backend.
Arguments: $ARGUMENTS (METHOD /path description)

Create files in functions/src/tutorMe/: Cloud Function handler, service,
Firestore schema types, and a test file.
Follow the existing patterns in functions/src/tutorMe/endpoints/.
Use { success, data, error } response format.
```

4.3 /write-test — Generate Tests

Analyses existing code and generates comprehensive tests:

```
Analyse the file at $ARGUMENTS and generate tests.
For C++ files: use UE5 Automation Framework (FAutomationTestBase).
For Cloud Functions: use Jest with Firebase emulators for Firestore tests.
Cover: happy path, edge cases, error handling, and boundary conditions.
Place test files alongside source with .test.js or Test prefix for C++.
```

4.4 /review-pr — AI Code Review

Reviews a pull request for quality, conventions, and bugs:

```
Review the current git diff or PR at $ARGUMENTS.
```

```
Check for: UE5 coding standard compliance, memory safety (raw pointers),
Blueprint exposure correctness, API contract adherence,
missing error handling, potential crashes, and test coverage.
Provide specific, actionable feedback with file:line references.
```

4.5 /implement-prd-section — Build from PRD

This is the most powerful command. It reads a section of the PRD and generates the full implementation:

```
Read the Tutor Me PRD section: $ARGUMENTS (e.g., 'FR-03 Teaching Phase')

Generate the complete implementation across all modules:
1. UE5 C++ classes needed (in Source/TutorMe/)
2. Backend API endpoints needed (in Backend/TutorMe/)
3. AI evaluation prompts needed (in AI/TutorMe/)
4. Data model changes needed
5. Tests for each component

Create a GitHub branch, commit the code, and prepare a PR.
Reference the PRD section in the PR description.
```

5. Sprint-by-Sprint Implementation Plan

Each sprint is 2 weeks. The plan shows what the dev team builds, which Claude Code commands they use, and the expected deliverables. Total timeline: 12 weeks (6 sprints) for Phase 1 MVP.

Sprint 1 (Weeks 1–2): Foundation & Data Layer

Role	Tasks	Claude Code Usage
Full-stack Dev	Add videoSegments collection to Firestore with segment-level fields. Create Cloud Function for GET /api/v1/videos/{id}/segments. Backfill segment data for 20 pilot videos.	Use /generate-api-endpoint to scaffold the Cloud Function. Use Claude Code to generate the Firestore schema and seed data scripts.
Backend Dev	Create tutorSessions collection in Firestore. Build Cloud Functions for POST /tutor-sessions and PATCH /tutor-sessions/{id}. Set up Firestore security rules.	Use /generate-api-endpoint for each Cloud Function. Claude Code generates the Firestore document schemas and security rules from the PRD data model.
UE5 Dev	Set up TutorMe C++ module in the UE5 project. Create base classes: UTutorMeSubsystem, FTutorMeSegmentData struct, UTutorMeSessionState.	Use /generate-ue-class for each class. Claude Code writes the USTRUCT for segment data matching the API schema exactly.

Deliverables: Working API that returns segment data. Session CRUD endpoints. UE5 module compiles with base classes.

Sprint 2 (Weeks 3–4): Tutor Me Button & Session Init

Role	Tasks	Claude Code Usage
UE5 Dev	Create UTutorMeButtonWidget (UMG C++ class). Add to video player overlay. Implement tap handler: pause video, store timestamp, trigger session init. Create UTutorMeSessionWidget (main session screen layout).	Use /implement-prd-section FR-01 to generate the button widget. Claude Code creates the full UMG C++ class with delegate bindings.
UE5 Dev	Build loading screen with Miss Lily avatar animation. Implement HTTP client to call GET /segments on session start. Parse JSON response into FTutorMeSegmentData array.	Claude Code writes the HTTP request/response handling using UE5's FHttpModule. Generates JSON deserialization code matching the API schema.

Backend Dev	Implement caching layer using Firebase's built-in caching and Cloud CDN. Optimise Firestore query performance with composite indexes. Add loading state endpoint.	Use Claude Code to generate Firestore composite index definitions and Cloud Function caching middleware.

Deliverables: Tapping "Tutor Me" on video pauses video and opens session. Segment data loads and displays.

Sprint 3 (Weeks 5–6): Teaching Phase & Miss Lily Integration

Role	Tasks	Claude Code Usage
UE5 Dev	Build segment teaching view: split layout with Miss Lily avatar area + content display area. Implement image carousel from segment data. Add progress bar (Segment X of Y). Wire up Miss Lily avatar controller to speak narration script.	Use /implement-prd-section FR-03 for the full teaching phase. Claude Code generates UTutorMeTeachingWidget, UTutorMeImageCarousel, and the segment progress indicator.
UE5 Dev	Integrate Text-to-Speech API for Miss Lily's voice. Implement lip-sync trigger system. Add "Repeat" and "Quiz me!" button actions.	Claude Code writes the TTS API client class and the avatar animation state machine in C++.
AI Engineer	Begin building evaluation prompt templates. Define key concept extraction logic. Create scoring rubric algorithm.	Claude Code generates the prompt templates in AI/TutorMe/prompts/ following few-shot patterns. Writes the scoring algorithm in Python.

Deliverables: Miss Lily teaches a segment with images, voice, and text. Student can repeat or proceed.

Sprint 4 (Weeks 7–8): Assessment & Evaluation Engine

Role	Tasks	Claude Code Usage
UE5 Dev	Build student input UI: voice recording (microphone), text input, send button. Integrate Speech-to-Text API. Display typing indicator while evaluation runs. Show evaluation result card (score ring, feedback, reward).	Use /implement-prd-section FR-04 and FR-05. Claude Code generates UTutorMeInputWidget with voice/text/handwriting modes and the evaluation result display widget.
Backend Dev	Build Cloud Function for POST /tutor-sessions/{id}/evaluate. Integrate with AI evaluation engine. Store	Use /generate-api-endpoint for the evaluate Cloud Function. Claude Code writes the service layer that calls the AI engine and updates Firestore.

	results in Firestore tutorSessions subcollection.	
AI Engineer	Implement full evaluation engine: coverage, accuracy, depth, coherence scoring. Build gap identification algorithm. Create age-appropriate feedback generator.	Claude Code writes the evaluation chain-of-thought prompts, the scoring rubric implementation, and the feedback templates for pass/fail scenarios.

Deliverables: Student can respond to a segment. AI evaluates and returns a score with specific feedback.

Sprint 5 (Weeks 9–10): Remediation & Segment Loop

Role	Tasks	Claude Code Usage
UE5 Dev	Implement the Teach→Assess→Evaluate→Remediate loop state machine. Build remediation UI: analogy cards, visual emphasis mode, simplified re-explanation. Track attempt count (max 3). Advance to next segment on pass.	Use /implement-prd-section FR-06. Claude Code generates the UTutorMeStateMachine class with all state transitions and the remediation widget variants.
Backend Dev	Build POST /remediate endpoint. Implement adaptive strategy selection (simplify, analogy, visual, breakdown, question-led). Track attempts in session model.	Claude Code generates the strategy selection algorithm and the remediation content recomposer that works within Smart Content Studio assets.
AI Engineer	Create 5 remediation prompt strategies. Build strategy selector based on gap type. Implement graduated hint system (attempt 2: partial hint, attempt 3: scaffold).	Claude Code writes all 5 strategy prompts as few-shot templates. Generates the hint graduation logic and the “weak area” flagging system.

Deliverables: Full segment loop works end-to-end. Student can fail, get remediation, retry, and advance.

Sprint 6 (Weeks 11–12): Final Assessment, Summary & Polish

Role	Tasks	Claude Code Usage
UE5 Dev	Build final assessment screen with full image gallery carousel. Implement session summary screen: mastery score, per-segment breakdown, XP/coin awards, streak tracking. Add	Use /implement-prd-section FR-07 and FR-08. Claude Code generates the final assessment widget, summary screen, and reward animation triggers.

	celebration animations. Wire up gamification rewards.	
Backend Dev	Build POST /final-evaluate and GET /summary endpoints. Integrate with Teacher Dashboard and Parent Dashboard APIs. Implement spaced repetition queue for weak concepts. XP/coin award logic.	Use /generate-api-endpoint for final endpoints. Claude Code writes the dashboard integration layer and the spaced repetition queue logic.
All Devs	End-to-end testing across all 5 pilot videos. Bug fixing. Performance optimisation. Edge case handling (network drops, empty responses, noisy environment). COPPA compliance audit.	Use /write-test to generate comprehensive test suites. Use /review-pr for all final PRs. Claude Code helps identify edge cases from the PRD.

Deliverables: Complete Tutor Me MVP. Full flow from video to session to summary. Tested on 5 pilot lessons.

6. GitHub Workflow with Claude Code

Here is how your GitHub branches and PRs should be structured for the Tutor Me feature:

6.1 Branch Strategy

```
main                              ← Production (protected)
  └─ develop                       ← Integration branch
      └─ feature/tutor-me           ← Feature branch (6 sprints merge here)
          ├─ tutor-me/ue5-button-widget        ← Sprint 2 UE5 work
          ├─ tutor-me/api-segments-endpoint    ← Sprint 1 backend work
          ├─ tutor-me/ai-evaluation-engine     ← Sprint 4 AI work
          └─ tutor-me/ue5-remediation-loop     ← Sprint 5 UE5 work
```

6.2 How Claude Code Creates PRs

When a developer finishes a task, they can say to Claude Code: “Create a PR for this work targeting feature/tutor-me.” Claude Code will:

1. Stage the relevant files (never staging .env or credentials)
2. Write a commit message following the [TutorMe][Module] convention
3. Push to the developer’s branch
4. Create a GitHub PR with: title, summary of changes, test plan, PRD section reference, and screenshots placeholder for UI changes
5. Request review from the appropriate team member

6.3 How Claude Code Reviews PRs

Any team member can ask Claude Code to review a PR: “Review PR #42.” Claude Code will check for UE5 coding standard compliance, memory safety, Blueprint exposure correctness, API contract adherence, and test coverage — then post specific feedback.

7. Getting Started — First Day Setup

Follow these steps to set up Claude Code for every developer on your team:

1. **Install Claude Code:** Each developer installs Claude Code CLI on their machine. Run: npm install -g @anthropic-ai/claude-code
2. **Clone the repo:** git clone your PTL AI GitHub repository.
3. **Add the CLAUDE.md file:** Copy the CLAUDE.md content from Section 3 into .claude/CLAUDE.md in the repo root.
4. **Add custom commands:** Create the .claude/commands/ folder and add the 5 command files from Section 4.
5. **Commit and push:** This is a one-time setup. Every developer who pulls the repo will automatically have Claude Code configured.

6. **Start the first sprint:** Open terminal in the repo. Type ‘claude’ to start Claude Code. Give it the first task from Sprint 1.

7.1 Example First Session

Here is what a developer’s first Claude Code session might look like for Sprint 1:

```
$ cd PTL-AI
$ claude

> Create the FTutorMeSegmentData struct in Source/TutorMe/Public/
  with all fields from the PRD Section 6.1 data schema.
  Include UPROPERTY macros for Blueprint access and JSON serialization.

[Claude Code reads CLAUDE.md, understands the project, generates the struct]

> Now create the UTutorMeAPIClient class that fetches segment data
  from GET /api/v1/videos/{video_id}/segments and deserialises
  the response into an array of FTutorMeSegmentData.

[Claude Code generates the HTTP client class with async callbacks]

> Create a PR for this work targeting feature/tutor-me
```

End of Document

For questions or to begin the Claude Code setup, contact Thiruvengadam at thiru@imercfy.com.